Bulletproofs: applications

October 29, 2026

Blockstream Research

research.blockstream.com

github.com/BlockstreamResearch/zk-book



Plan

1 Introduction

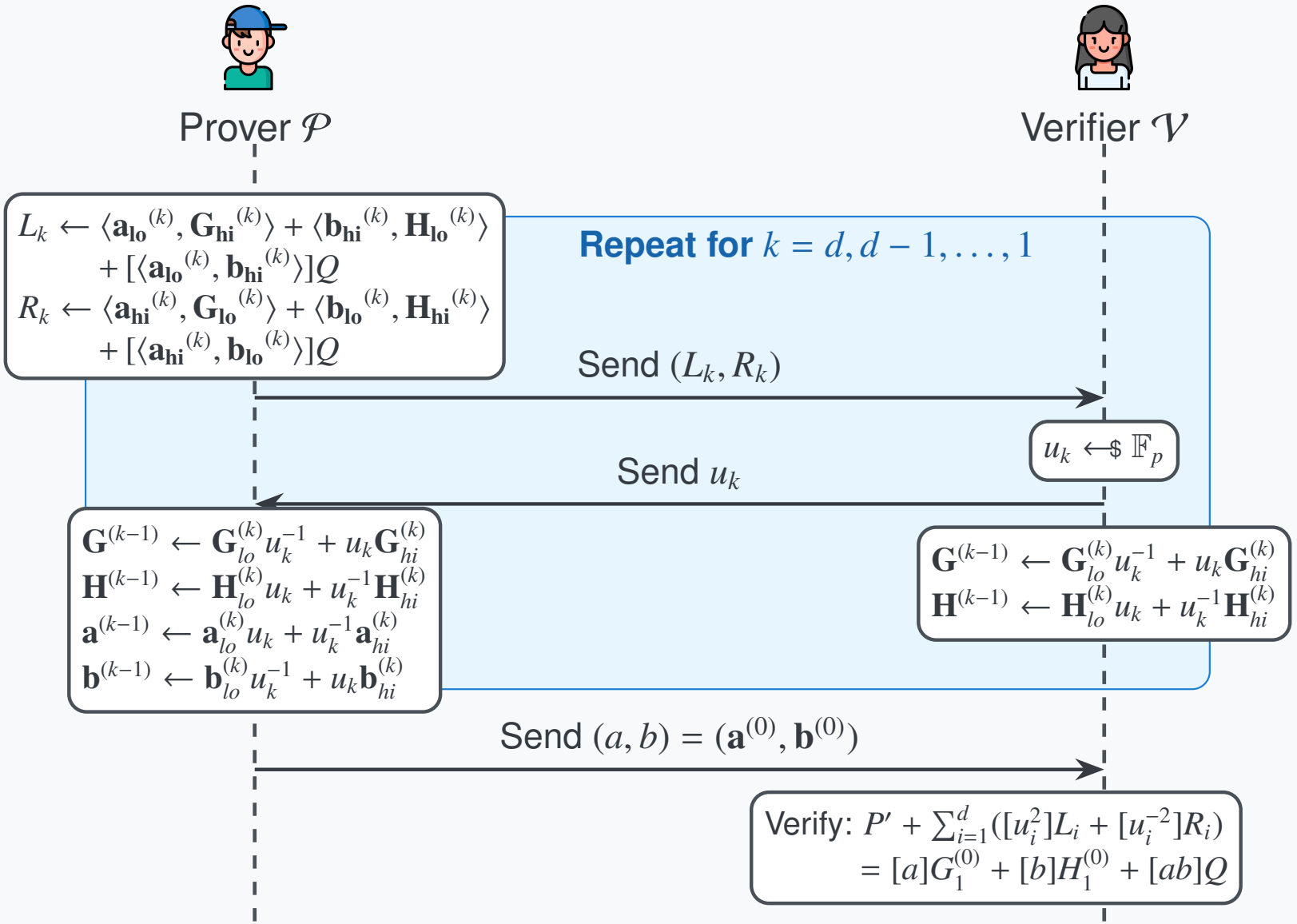
2 IPA polynomial commitment scheme

3 Range proofs

4 Arithmetic circuits

Introduction

Inner-product argument: illustration



Recap: inner-product argument

- **Goal:** Prove $\langle \mathbf{a}, \mathbf{b} \rangle = c$ with logarithmic proof size
- Commitment: $P' = \langle \mathbf{a}, \mathbf{G} \rangle + \langle \mathbf{b}, \mathbf{H} \rangle + [\langle \mathbf{a}, \mathbf{b} \rangle] Q$
- Protocol recursively compresses vectors at each step
- Final check: $P' + \sum_{i=1}^{d} ([u_i^2] L_i + [u_i^{-2}] R_i) = [a] G_1^{(0)} + [b] H_1^{(0)} + [ab] Q$

Recap: inner-product argument

- **Goal:** Prove $\langle \mathbf{a}, \mathbf{b} \rangle = c$ with logarithmic proof size
- Commitment: $P' = \langle \mathbf{a}, \mathbf{G} \rangle + \langle \mathbf{b}, \mathbf{H} \rangle + [\langle \mathbf{a}, \mathbf{b} \rangle] Q$
- Protocol recursively compresses vectors at each step
- Final check: $P' + \sum_{i=1}^{d} ([u_i^2] L_i + [u_i^{-2}] R_i) = [a] G_1^{(0)} + [b] H_1^{(0)} + [ab] Q$

Key properties

Proof size is $O(\log_2 n)$, prover and verifier both run in $O(n)$. The protocol doesn't need a *trusted setup*. Protocol is *knowledge sound* and *perfect complete* but not *zero-knowledge*.

Recap: inner-product argument

- **Goal:** Prove $\langle \mathbf{a}, \mathbf{b} \rangle = c$ with logarithmic proof size
- Commitment: $P' = \langle \mathbf{a}, \mathbf{G} \rangle + \langle \mathbf{b}, \mathbf{H} \rangle + [\langle \mathbf{a}, \mathbf{b} \rangle] Q$
- Protocol recursively compresses vectors at each step
- Final check: $P' + \sum_{i=1}^{d} ([u_i^2] L_i + [u_i^{-2}] R_i) = [a] G_1^{(0)} + [b] H_1^{(0)} + [ab] Q$

Key properties

Proof size is $O(\log_2 n)$, prover and verifier both run in $O(n)$. The protocol doesn't need a *trusted setup*. Protocol is *knowledge sound* and *perfect complete* but not *zero-knowledge*.

Idea

We could provide *zero-knowledge* directly to *inner-product argument* construction or use **zk-mul** protocol for outer construction.

Recap: zk-mul

Consider relation $\mathcal{R}_{mul} = \{(\perp; l(x), r(x), t(x)) : t(x) = l(x)r(x)\}$ where $l(x) = a + s_L x, r(x) = b + s_R x, t(x) = l(x)r(x)$. Protocol **zk-mul** ($\Pi_{mul}$) is defined as follows:

- Prover computes and sends to $\mathcal{V}$ commitments to $l(x), r(x), t(x)$:

$$A = [a]G + [b]H + [\alpha]B \qquad T_0 = [ab]G + [\tau_0]B$$
$$S = [s_L]G + [s_R]H + [\beta]B \qquad T_1 = [as_R + bs_L]G + [\tau_1]B$$
$$T_2 = [s_L s_R]G + [\tau_2]B$$

Recap: zk-mul

Consider relation $\mathcal{R}_{mul} = \{(\perp; l(x), r(x), t(x)) : t(x) = l(x)r(x)\}$ where $l(x) = a + s_L x, r(x) = b + s_R x, t(x) = l(x)r(x)$. Protocol **zk-mul** ($\Pi_{mul}$) is defined as follows:

- Prover computes and sends to $\mathcal{V}$ commitments to $l(x), r(x), t(x)$:

$$A = [a]G + [b]H + [\alpha]B \qquad T_0 = [ab]G + [\tau_0]B$$
$$S = [s_L]G + [s_R]H + [\beta]B \qquad T_1 = [as_R + bs_L]G + [\tau_1]B$$
$$T_2 = [s_L s_R]G + [\tau_2]B$$

- Verifier draws random challenge $u \leftarrow\$ \mathbb{F}_p$ and sends it to prover

Recap: zk-mul

Consider relation $\mathcal{R}_{mul} = \{(\perp; l(x), r(x), t(x)) : t(x) = l(x)r(x)\}$ where $l(x) = a + s_L x, r(x) = b + s_R x, t(x) = l(x)r(x)$. Protocol **zk-mul** ($\Pi_{mul}$) is defined as follows:

- Prover computes and sends to $\mathcal{V}$ commitments to $l(x), r(x), t(x)$:

$$A = [a]G + [b]H + [\alpha]B \qquad T_0 = [ab]G + [\tau_0]B$$
$$S = [s_L]G + [s_R]H + [\beta]B \qquad T_1 = [as_R + bs_L]G + [\tau_1]B$$
$$T_2 = [s_L s_R]G + [\tau_2]B$$

- Verifier draws random challenge $u \leftarrow\$ \mathbb{F}_p$ and sends it to prover
- Prover evaluates and sends to Verifier $(l_u, r_u, t_u, \alpha_u, \tau_u)$:

$$l_u = l(u),\ r_u = r(u),\ t_u = l_u \cdot r_u,\ \alpha_u = \alpha + \beta u,\ \tau_u = \tau_0 + \tau_1 u + \tau_2 u^2$$

Recap: zk-mul

Consider relation $\mathcal{R}_{mul} = \{(\perp; l(x), r(x), t(x)) : t(x) = l(x)r(x)\}$ where $l(x) = a + s_L x, r(x) = b + s_R x, t(x) = l(x)r(x)$. Protocol **zk-mul** ($\Pi_{mul}$) is defined as follows:

- Prover computes and sends to $\mathcal{V}$ commitments to $l(x), r(x), t(x)$:

$$A = [a]G + [b]H + [\alpha]B \qquad T_0 = [ab]G + [\tau_0]B$$
$$S = [s_L]G + [s_R]H + [\beta]B \qquad T_1 = [as_R + bs_L]G + [\tau_1]B$$
$$T_2 = [s_L s_R]G + [\tau_2]B$$

- Verifier draws random challenge $u \leftarrow\$ \mathbb{F}_p$ and sends it to prover
- Prover evaluates and sends to Verifier $(l_u, r_u, t_u, \alpha_u, \tau_u)$:

$$l_u = l(u),\ r_u = r(u),\ t_u = l_u \cdot r_u,\ \alpha_u = \alpha + \beta u,\ \tau_u = \tau_0 + \tau_1 u + \tau_2 u^2$$

- Verifier checks: $A + [u]S \stackrel{?}{=} [l_u]G + [r_u]H + [\alpha_u]B$, $[t_u]G + [\tau_u]B \stackrel{?}{=} T_0 + [u]T_1 + [u^2]T_2$, $t_u \stackrel{?}{=} l_u r_u$

What's next?


IPA
zk-mul
Range proofs &
Arithmetic circuits
@Petirep
JAKE-CLARK.TUMBLR
imgflip.com

IPA polynomial commitment scheme

Recap: Polynomial commitments

- **Polynomial commitment scheme**

$$C = (\text{Setup}, \text{Commit}, \text{Open}, \text{VerifyOpen})$$

allows to commit to a polynomial $f(x) = \sum_{i=0}^{n-1} a_i x^i$ and prove its evaluation at some point.

Recap: Polynomial commitments

- **Polynomial commitment scheme**

$$C = (\text{Setup}, \text{Commit}, \text{Open}, \text{VerifyOpen})$$

  allows to commit to a polynomial $f(x) = \sum_{i=0}^{n-1} a_i x^i$ and prove its evaluation at some point.

- **Applications:** SNARKs compiled with IOP + polynomial commitment scheme framework (e.g., Halo, Nova, Spartan, Plonk)

Recap: Polynomial commitments

- **Polynomial commitment scheme**

$$C = (\text{Setup}, \text{Commit}, \text{Open}, \text{VerifyOpen})$$

  allows to commit to a polynomial $f(x) = \sum_{i=0}^{n-1} a_i x^i$ and prove its evaluation at some point.

- **Applications:** SNARKs compiled with IOP + polynomial commitment scheme framework (e.g., Halo, Nova, Spartan, Plonk)
- **Desirable properties:** sublinear size, efficient, no trusted setup

Recap: Polynomial commitments

- **Polynomial commitment scheme**

$$C = (\text{Setup}, \text{Commit}, \text{Open}, \text{VerifyOpen})$$

  allows to commit to a polynomial $f(x) = \sum_{i=0}^{n-1} a_i x^i$ and prove its evaluation at some point.

- **Applications:** SNARKs compiled with IOP + polynomial commitment scheme framework (e.g., Halo, Nova, Spartan, Plonk)
- **Desirable properties:** sublinear size, efficient, no trusted setup

Example

One famous example is the **KZG** polynomial commitment scheme, which uses bilinear pairings and requires a trusted setup.

IPA polynomial commitment

Let $f(x) = \sum_{i=0}^{n-1} a_i x^i \in \mathbb{F}_p[x]$ be a polynomial of degree $n - 1 = 2^d - 1$.

The **non-hiding IPA polynomial commitment scheme** $C_{ip} = (\text{Setup}, \text{Commit}, \text{Open}, \text{VerifyOpen})$ is defined as follows:

- Setup returns independent generators $\mathbf{G} = (G_1, \ldots, G_n)$.

IPA polynomial commitment

Let $f(x) = \sum_{i=0}^{n-1} a_i x^i \in \mathbb{F}_p[x]$ be a polynomial of degree $n - 1 = 2^d - 1$.

The **non-hiding IPA polynomial commitment scheme** $C_{ip} = (\text{Setup}, \text{Commit}, \text{Open}, \text{VerifyOpen})$ is defined as follows:

- Setup returns independent generators $\mathbf{G} = (G_1, \ldots, G_n)$.
- Commit returns $\text{Com}(f) = \langle \mathbf{f}, \mathbf{G} \rangle$ where $\mathbf{f} = (a_0, \ldots, a_{n-1})$

IPA polynomial commitment

Let $f(x) = \sum_{i=0}^{n-1} a_i x^i \in \mathbb{F}_p[x]$ be a polynomial of degree $n - 1 = 2^d - 1$.

The **non-hiding IPA polynomial commitment scheme** $C_{ip} = (\text{Setup}, \text{Commit}, \text{Open}, \text{VerifyOpen})$ is defined as follows:

- Setup returns independent generators $\mathbf{G} = (G_1, \ldots, G_n)$.
- Commit returns $\text{Com}(f) = \langle \mathbf{f}, \mathbf{G} \rangle$ where $\mathbf{f} = (a_0, \ldots, a_{n-1})$
- Open given evaluation point $u \in \mathbb{F}_p$ computes $\mathbf{u}^n = (1, u, u^2, \ldots, u^{n-1})$, obtains $f(u) = \langle \mathbf{f}, \mathbf{u}^n \rangle$ and runs *inner-product argument* Π_{ip} non-interactively setting

$$\mathbf{a} = \mathbf{f}, \mathbf{b} = \mathbf{u}^n, P = \text{Com}(f), c = f(u)$$

to produce an evaluation proof π_{ip}

IPA polynomial commitment

Let $f(x) = \sum_{i=0}^{n-1} a_i x^i \in \mathbb{F}_p[x]$ be a polynomial of degree $n - 1 = 2^d - 1$.

The **non-hiding IPA polynomial commitment scheme** $C_{ip} = (\text{Setup}, \text{Commit}, \text{Open}, \text{VerifyOpen})$ is defined as follows:

- Setup returns independent generators $\mathbf{G} = (G_1, \ldots, G_n)$.
- Commit returns $\text{Com}(f) = \langle \mathbf{f}, \mathbf{G} \rangle$ where $\mathbf{f} = (a_0, \ldots, a_{n-1})$
- Open given evaluation point $u \in \mathbb{F}_p$ computes $\mathbf{u}^n = (1, u, u^2, \ldots, u^{n-1})$, obtains $f(u) = \langle \mathbf{f}, \mathbf{u}^n \rangle$ and runs *inner-product argument* Π_{ip} non-interactively setting

$$\mathbf{a} = \mathbf{f}, \mathbf{b} = \mathbf{u}^n, P = \text{Com}(f), c = f(u)$$

  to produce an evaluation proof π_{ip}
- VerifyOpen given evaluation point $u \in \mathbb{F}_p$ and commitment $\text{Com}(f)$ validates proof π_{ip} running the non-interactive verifier $\mathcal{V}$ of *inner-product* argument.

Range proofs

Range proofs: motivation

- **Goal:** Prove $v \in [0, 2^n)$ without revealing v:

$$\mathcal{R}_{rp} = \{(G, B, V, n; v, \gamma) : V = [v]G + [\gamma]B, v \in [0, 2^n)\}$$

Range proofs: motivation

- **Goal:** Prove $v \in [0, 2^n)$ without revealing v:

$$\mathcal{R}_{rp} = \{(G, B, V, n; v, \gamma) : V = [v]G + [\gamma]B, v \in [0, 2^n)\}$$

- **Applications:** Confidential transactions (e.g., Monero, Mimblewimble), other privacy-preserving protocols.

Range proofs: motivation

- **Goal:** Prove $v \in [0, 2^n)$ without revealing v:

$$\mathcal{R}_{rp} = \{(G, B, V, n; v, \gamma) : V = [v]G + [\gamma]B, v \in [0, 2^n)\}$$

- **Applications:** Confidential transactions (e.g., Monero, Mimblewimble), other privacy-preserving protocols.
- **Idea**: Prove $v = \sum_{i=0}^{n-1} v_i 2^i$ and $v_i \in \{0, 1\}$ for all $i \in [n]$

Range proofs: motivation

- **Goal:** Prove $v \in [0, 2^n)$ without revealing v:

$$\mathcal{R}_{rp} = \{(G, B, V, n; v, \gamma) : V = [v]G + [\gamma]B, v \in [0, 2^n)\}$$

- **Applications:** Confidential transactions (e.g., Monero, Mimblewimble), other privacy-preserving protocols.
- **Idea**: Prove $v = \sum_{i=0}^{n-1} v_i 2^i$ and $v_i \in \{0, 1\}$ for all $i \in [n]$

Naive approach

One could prove $v = \sum_{i=0}^{n-1} v_i 2^i$ and $v_i \in \{0, 1\}$ for all $i \in [n]$ using Σ-protocols, but this would be inefficient due to linear proof.

Compiling range proof into inner-product

Firstly, write v in base-2 representation: $v = \sum_{i=0}^{\lfloor \log_2 v \rfloor} 2^i v_i$ and $\mathbf{a}_L = (v_0, v_1, \ldots, v_{n-1})$ be the vector of bits padded with zeroes to length n, define $\mathbf{a}_R = \mathbf{a}_L - \mathbf{1}^n$ so the range validation that v lies in $[0, 2^n)$ implies two checks:

Compiling range proof into inner-product

Firstly, write v in base-2 representation: $v = \sum_{i=0}^{\lfloor \log_2 v \rfloor} 2^i v_i$ and $\mathbf{a}_L = (v_0, v_1, \ldots, v_{n-1})$ be the vector of bits padded with zeroes to length n, define $\mathbf{a}_R = \mathbf{a}_L - \mathbf{1}^n$ so the range validation that v lies in $[0, 2^n)$ implies two checks:

- The following inner-product equality holds: $\langle \mathbf{a}_L, \mathbf{2}^n \rangle = v$

Compiling range proof into inner-product

Firstly, write v in base-2 representation: $v = \sum_{i=0}^{\lfloor \log_2 v \rfloor} 2^i v_i$ and $\mathbf{a}_L = (v_0, v_1, \ldots, v_{n-1})$ be the vector of bits padded with zeroes to length n, define $\mathbf{a}_R = \mathbf{a}_L - \mathbf{1}^n$ so the range validation that v lies in $[0, 2^n)$ implies two checks:

- The following inner-product equality holds: $\langle \mathbf{a}_L, \mathbf{2}^n \rangle = v$
- Each bit v_i must be either 0 or 1:

$$\mathbf{a}_L - \mathbf{a}_R - \mathbf{1}^n = \mathbf{0}^n$$
$$\mathbf{a}_L \circ \mathbf{a}_R = \mathbf{0}^n$$

Compiling range proof into inner-product

Firstly, write v in base-2 representation: $v = \sum_{i=0}^{\lfloor \log_2 v \rfloor} 2^i v_i$ and $\mathbf{a}_L = (v_0, v_1, \ldots, v_{n-1})$ be the vector of bits padded with zeroes to length n, define $\mathbf{a}_R = \mathbf{a}_L - \mathbf{1}^n$ so the range validation that v lies in $[0, 2^n)$ implies two checks:

- The following inner-product equality holds: $\langle \mathbf{a}_L, \mathbf{2}^n \rangle = v$
- Each bit v_i must be either 0 or 1:

$$\mathbf{a}_L - \mathbf{a}_R - \mathbf{1}^n = \mathbf{0}^n$$
$$\mathbf{a}_L \circ \mathbf{a}_R = \mathbf{0}^n$$

Example

Let $\mathbf{a}_L = (1, 0, 1, 0)$, $\mathbf{a}_R = (0, -1, 0, -1)$, then $\mathbf{a}_L \circ \mathbf{a}_R = (0, 0, 0, 0)$

Compiling range proof into inner-product

These two checks imply verification that some vector is zero vector, for that we use some challenge $y \in \mathbb{F}_p$ and check inner-product equalities

$$\langle \mathbf{a}_L \circ \mathbf{a}_R, \mathbf{y}^n \rangle = 0 \text{ and } \langle \mathbf{a}_L - \mathbf{a}_R - \mathbf{1}^n, \mathbf{y}^n \rangle = 0$$

These checks are sound because the prover doesn't know challenge y in advance.

Compiling range proof into inner-product

These two checks imply verification that some vector is zero vector, for that we use some challenge $y \in \mathbb{F}_p$ and check inner-product equalities

$$\langle \mathbf{a}_L \circ \mathbf{a}_R, \mathbf{y}^n \rangle = 0 \text{ and } \langle \mathbf{a}_L - \mathbf{a}_R - \mathbf{1}^n, \mathbf{y}^n \rangle = 0$$

These checks are sound because the prover doesn't know challenge y in advance. So we must combine three inner-product checks:

1. $\langle \mathbf{a}_L, \mathbf{2}^n \rangle = v$
2. $\langle \mathbf{a}_L, \mathbf{a}_R \circ \mathbf{y}^n \rangle = 0$
3. $\langle \mathbf{a}_L - \mathbf{a}_R - \mathbf{1}^n, \mathbf{y}^n \rangle = 0$

Compiling range proof into inner-product

These two checks imply verification that some vector is zero vector, for that we use some challenge $y \in \mathbb{F}_p$ and check inner-product equalities

$$\langle \mathbf{a}_L \circ \mathbf{a}_R, \mathbf{y}^n \rangle = 0 \text{ and } \langle \mathbf{a}_L - \mathbf{a}_R - \mathbf{1}^n, \mathbf{y}^n \rangle = 0$$

These checks are sound because the prover doesn't know challenge y in advance. So we must combine three inner-product checks:

1. $\langle \mathbf{a}_L, \mathbf{2}^n \rangle = v$
2. $\langle \mathbf{a}_L, \mathbf{a}_R \circ \mathbf{y}^n \rangle = 0$
3. $\langle \mathbf{a}_L - \mathbf{a}_R - \mathbf{1}^n, \mathbf{y}^n \rangle = 0$

into one soundly summing up with powers of other challenge $z \in \mathbb{F}_p$:

$$z^2 \cdot \langle \mathbf{a}_L, \mathbf{2}^n \rangle + z \cdot \langle \mathbf{a}_L - \mathbf{a}_R - \mathbf{1}^n, \mathbf{y}^n \rangle + \langle \mathbf{a}_L, \mathbf{a}_R \circ \mathbf{y}^n \rangle = z^2 v$$

Compiling range proof into inner-product

Using some dark linear algebra wizardry we could combine the three inner-product checks into a single one inner-product check:

$$\langle \mathbf{a}_L - z \cdot \mathbf{1}^n, z^2 \cdot \mathbf{2}^n + z \cdot \mathbf{y}^n + \mathbf{a}_R \circ \mathbf{y}^n \rangle = z^2 v + \delta(y, z)$$

Where $\delta(y, z)$ could easily be computed by verifier:

$$\delta(y, z) = (z - z^2)\langle \mathbf{1}^n, \mathbf{y}^n \rangle - z^3 \langle \mathbf{1}^n, \mathbf{2}^n \rangle$$

Compiling range proof into inner-product

Using some dark linear algebra wizardry we could combine the three inner-product checks into a single one inner-product check:

$$\langle \mathbf{a}_L - z \cdot \mathbf{1}^n, z^2 \cdot \mathbf{2}^n + z \cdot \mathbf{y}^n + \mathbf{a}_R \circ \mathbf{y}^n \rangle = z^2 v + \delta(y, z)$$

Where $\delta(y, z)$ could easily be computed by verifier:

$$\delta(y, z) = (z - z^2)\langle \mathbf{1}^n, \mathbf{y}^n \rangle - z^3 \langle \mathbf{1}^n, \mathbf{2}^n \rangle$$

Now it's time to bring out **zk-mul** for inner-products!

Firstly, construct the blinding polynomials for $\mathbf{a}_L$ and $\mathbf{a}_R$:

$$\mathbf{a}'_L \leftarrow \mathbf{a}_L + \mathbf{s}_L x \quad \mathbf{a}'_R \leftarrow \mathbf{a}_R + \mathbf{s}_R x$$

Compiling range proof into inner-product

Using some dark linear algebra wizardry we could combine the three inner-product checks into a single one inner-product check:

$$\langle \mathbf{a}_L - z \cdot \mathbf{1}^n, z^2 \cdot \mathbf{2}^n + z \cdot \mathbf{y}^n + \mathbf{a}_R \circ \mathbf{y}^n \rangle = z^2 v + \delta(y, z)$$

Where $\delta(y, z)$ could easily be computed by verifier:

$$\delta(y, z) = (z - z^2)\langle \mathbf{1}^n, \mathbf{y}^n \rangle - z^3 \langle \mathbf{1}^n, \mathbf{2}^n \rangle$$

Now it's time to bring out **zk-mul** for inner-products!

Firstly, construct the blinding polynomials for $\mathbf{a}_L$ and $\mathbf{a}_R$:

$$\mathbf{a}'_L \leftarrow \mathbf{a}_L + \mathbf{s}_L x \quad \mathbf{a}'_R \leftarrow \mathbf{a}_R + \mathbf{s}_R x$$

Compute polynomials $\mathbf{l}(x) = \mathbf{l}_0 + \mathbf{l}_1 x, \quad \mathbf{r}(x) = \mathbf{r}_0 + \mathbf{r}_1 x$:

$$\mathbf{l}(x) = \mathbf{a}'_L - z \cdot \mathbf{1}^n = (\mathbf{a}_L + \mathbf{s}_L x) - z \cdot \mathbf{1}^n = \mathbf{a}_L - z \cdot \mathbf{1}^n + \mathbf{s}_L x$$

$$\mathbf{r}(x) = z^2 \cdot \mathbf{2}^n + z \cdot \mathbf{y}^n + \mathbf{a}'_R \circ \mathbf{y}^n = z^2 \cdot \mathbf{2}^n + z \cdot \mathbf{y}^n + (\mathbf{a}_R + \mathbf{s}_R x) \circ \mathbf{y}^n$$

$$= z^2 \cdot \mathbf{2}^n + z \cdot \mathbf{y}^n + \mathbf{a}_R \circ \mathbf{y}^n + \mathbf{s}_R \circ \mathbf{y}^n x$$

Compiling range proof into inner-product

$$t(x) = \langle \mathbf{l}(x), \mathbf{r}(x) \rangle = t_0 + t_1 x + t_2 x^2$$

Now $\mathcal{P}$ needs to apply **zk-mul** for proving:

$$t_0 = \langle \mathbf{a}_L - z \cdot \mathbf{1}^n, z^2 \cdot \mathbf{2}^n + z \cdot \mathbf{y}^n + \mathbf{a}_R \circ \mathbf{y}^n \rangle = z^2 v + \delta(y, z)$$

Compiling range proof into inner-product

$$t(x) = \langle \mathbf{l}(x), \mathbf{r}(x) \rangle = t_0 + t_1 x + t_2 x^2$$

Now $\mathcal{P}$ needs to apply **zk-mul** for proving:

$$t_0 = \langle \mathbf{a}_L - z \cdot \mathbf{1}^n, z^2 \cdot \mathbf{2}^n + z \cdot \mathbf{y}^n + \mathbf{a}_R \circ \mathbf{y}^n \rangle = z^2 v + \delta(y, z)$$

Note: $\mathcal{V}$ could compute commitment $\mathrm{Com}(t_0)$ using $V = \mathrm{Com}(v)$.

Compiling range proof into inner-product

$$t(x) = \langle \mathbf{l}(x), \mathbf{r}(x) \rangle = t_0 + t_1 x + t_2 x^2$$

Now $\mathcal{P}$ needs to apply **zk-mul** for proving:

$$t_0 = \langle \mathbf{a}_L - z \cdot \mathbf{1}^n, z^2 \cdot \mathbf{2}^n + z \cdot \mathbf{y}^n + \mathbf{a}_R \circ \mathbf{y}^n \rangle = z^2 v + \delta(y, z)$$

Note: $\mathcal{V}$ could compute commitment $\mathrm{Com}(t_0)$ using $V = \mathrm{Com}(v)$.

Remark

We couldn't apply raw **zk-mul** as $\mathbf{l}_0$ depends on verifier-provided challenges, instead $\mathcal{P}$ firstly commits to $\mathbf{a}_L, \mathbf{a}_R$ and blinders $\mathbf{s}_L, \mathbf{s}_R$, obtains challenges y, z from $\mathcal{V}$ and computes rest of the commitments.
During verification phase $\mathcal{V}$ should adjust commitments to $\mathbf{l}(x), \mathbf{r}(x)$ by himself using homomorphic properties of Pedersen commitment scheme.

Range proofs: building the protocol

- Setup returns independent generators $\mathbf{G}, \mathbf{H} \in \mathbb{G}^n$

Range proofs: building the protocol

- Setup returns independent generators $\mathbf{G}, \mathbf{H} \in \mathbb{G}^n$
- Prover does bit decomposition of v: $\mathbf{a}_L \leftarrow \mathbf{v}, \mathbf{a}_R \leftarrow \mathbf{a}_L - \mathbf{1}^n$, chooses blinding terms $\mathbf{s}_L, \mathbf{s}_R \in \mathbb{F}_p^n, \alpha, \beta \in \mathbb{F}_p$, sends commitments:

$$A = \langle \mathbf{a}_L, \mathbf{G} \rangle + \langle \mathbf{a}_R, \mathbf{H} \rangle + [\alpha]B \qquad S = \langle \mathbf{s}_L, \mathbf{G} \rangle + \langle \mathbf{s}_R, \mathbf{H} \rangle + [\beta]B$$

Range proofs: building the protocol

- Setup returns independent generators $\mathbf{G}, \mathbf{H} \in \mathbb{G}^n$
- Prover does bit decomposition of v: $\mathbf{a}_L \leftarrow \mathbf{v}, \mathbf{a}_R \leftarrow \mathbf{a}_L - \mathbf{1}^n$, chooses blinding terms $\mathbf{s}_L, \mathbf{s}_R \in \mathbb{F}_p^n, \alpha, \beta \in \mathbb{F}_p$, sends commitments:

$$A = \langle \mathbf{a}_L, \mathbf{G} \rangle + \langle \mathbf{a}_R, \mathbf{H} \rangle + [\alpha]B \qquad S = \langle \mathbf{s}_L, \mathbf{G} \rangle + \langle \mathbf{s}_R, \mathbf{H} \rangle + [\beta]B$$

- Verifier $\mathcal{V}$ samples challenges $y, z \leftarrow\!\!\$\ \mathbb{F}_p$ and sends them to $\mathcal{P}$

Range proofs: building the protocol

- Setup returns independent generators $\mathbf{G}, \mathbf{H} \in \mathbb{G}^n$
- Prover does bit decomposition of v: $\mathbf{a}_L \leftarrow \mathbf{v}, \mathbf{a}_R \leftarrow \mathbf{a}_L - \mathbf{1}^n$, chooses blinding terms $\mathbf{s}_L, \mathbf{s}_R \in \mathbb{F}_p^n, \alpha, \beta \in \mathbb{F}_p$, sends commitments:

$$A = \langle \mathbf{a}_L, \mathbf{G} \rangle + \langle \mathbf{a}_R, \mathbf{H} \rangle + [\alpha]B \qquad S = \langle \mathbf{s}_L, \mathbf{G} \rangle + \langle \mathbf{s}_R, \mathbf{H} \rangle + [\beta]B$$

- Verifier $\mathcal{V}$ samples challenges $y, z \leftarrow\!\$\ \mathbb{F}_p$ and sends them to $\mathcal{P}$
- Prover $\mathcal{P}$ reconstructs polynomials $\mathbf{l}(x), \mathbf{r}(x), t(x)$:

$$\mathbf{l}(x) = \mathbf{a}_L - z \cdot \mathbf{1}^n + \mathbf{s}_L x$$

$$\mathbf{r}(x) = z^2 \cdot \mathbf{2}^n + z \cdot \mathbf{y}^n + \mathbf{a}_R \circ \mathbf{y}^n + \mathbf{s}_R \circ \mathbf{y}^n x$$

$$t(x) = \langle \mathbf{l}(x), \mathbf{r}(x) \rangle = t_0 + t_1 x + t_2 x^2$$

$$t_0 = \langle \mathbf{a}_L - z \cdot \mathbf{1}^n, z^2 \cdot \mathbf{2}^n + z \cdot \mathbf{y}^n + \mathbf{a}_R \circ \mathbf{y}^n \rangle = z^2 v + \delta(y, z)$$

$$t_1 = \langle \mathbf{a}_L - z \cdot \mathbf{1}^n, \mathbf{y}^n \circ \mathbf{s}_R \rangle + \langle \mathbf{y}^n \circ (\mathbf{a}_R + z \cdot \mathbf{1}^n) + z^2 \cdot \mathbf{2}^n, \mathbf{s}_L \rangle$$

$$t_2 = \langle \mathbf{s}_L, \mathbf{y}^n \circ \mathbf{s}_R \rangle$$

Range proofs: proving

- Prover $\mathcal{P}$ draws blinding factors $\tau_1, \tau_2 \leftarrow\$ \mathbb{F}_p$ and sends to $\mathcal{V}$ commitments for coefficients of $t(x)$:

$$T_1 = [t_1]G + [\tau_1]B$$
$$T_2 = [t_2]G + [\tau_2]B$$

Note: prover does not have to send commitment to t_0 as it's the inner-product we want to prove and it could be computed from high-level commitment V.

Range proofs: proving

- Prover $\mathcal{P}$ draws blinding factors $\tau_1, \tau_2 \leftarrow\$ \mathbb{F}_p$ and sends to $\mathcal{V}$ commitments for coefficients of $t(x)$:

$$T_1 = [t_1]G + [\tau_1]B$$
$$T_2 = [t_2]G + [\tau_2]B$$

  Note: prover does not have to send commitment to t_0 as it's the inner-product we want to prove and it could be computed from high-level commitment V.

- Verifier $\mathcal{V}$ samples and sends to $\mathcal{P}$ evaluation point $u \leftarrow\$ \mathbb{F}_p$

Range proofs: proving

- Prover $\mathcal{P}$ draws blinding factors $\tau_1, \tau_2 \leftarrow\$ \mathbb{F}_p$ and sends to $\mathcal{V}$ commitments for coefficients of $t(x)$:

$$T_1 = [t_1]G + [\tau_1]B$$
$$T_2 = [t_2]G + [\tau_2]B$$

  Note: prover does not have to send commitment to t_0 as it's the inner-product we want to prove and it could be computed from high-level commitment V.

- Verifier $\mathcal{V}$ samples and sends to $\mathcal{P}$ evaluation point $u \leftarrow\$ \mathbb{F}_p$
- Prover $\mathcal{P}$ evaluates polynomials at u:

$$\mathbf{l}_u = \mathbf{l}(u) \qquad \alpha_u = \alpha + \beta u$$
$$\mathbf{r}_u = \mathbf{r}(u) \qquad \tau_u = z^2\gamma + \tau_1 u + \tau_2 u^2$$
$$t_u = t(u) = t_0 + t_1 u + t_2 u^2$$

  and sends $(\mathbf{l}_u, \mathbf{r}_u, t_u, \alpha_u, \tau_u)$ to $\mathcal{V}$.

Range proofs: verification

- Verifier $\mathcal{V}$ checks:

$$A + [u]S + \langle -z \cdot \mathbf{1}^n, \mathbf{G} \rangle + \langle z \cdot \mathbf{y}^n + z^2 \cdot \mathbf{2}^n, \mathbf{y}^{-n} \circ \mathbf{H} \rangle$$

$$\stackrel{?}{=} \langle \mathbf{l}_u, \mathbf{G} \rangle + \langle \mathbf{r}_u, \mathbf{y}^{-n} \circ \mathbf{H} \rangle + [\alpha_u]B$$

$$[t_u]G + [\tau_u]B \stackrel{?}{=} [z^2]V + [\delta(y,z)]G + [u]T_1 + [u^2]T_2$$

$$t_u \stackrel{?}{=} \langle \mathbf{l}_u, \mathbf{r}_u \rangle$$

Range proofs: verification

- Verifier $\mathcal{V}$ checks:

$$A + [u]S + \langle -z \cdot \mathbf{1}^n, \mathbf{G} \rangle + \langle z \cdot \mathbf{y}^n + z^2 \cdot \mathbf{2}^n, \mathbf{y}^{-n} \circ \mathbf{H} \rangle$$
$$\stackrel{?}{=} \langle \mathbf{l}_u, \mathbf{G} \rangle + \langle \mathbf{r}_u, \mathbf{y}^{-n} \circ \mathbf{H} \rangle + [\alpha_u]B$$
$$[t_u]G + [\tau_u]B \stackrel{?}{=} [z^2]V + [\delta(y,z)]G + [u]T_1 + [u^2]T_2$$
$$t_u \stackrel{?}{=} \langle \mathbf{l}_u, \mathbf{r}_u \rangle$$

Remark

To provide logarithmic size-proof instead of sending $\mathbf{l}_u, \mathbf{r}_u$ parties could run an inner-product argument **IPA** on inputs $(\mathbf{G}, \mathbf{y}^{-n} \circ \mathbf{H}, P, t_u; \mathbf{l}_u, \mathbf{r}_u)$ where:

$$P = A + [u]S + \langle -z \cdot \mathbf{1}^n, \mathbf{G} \rangle + \langle z \cdot \mathbf{y}^n + z^2 \cdot \mathbf{2}^n, \mathbf{y}^{-n} \circ \mathbf{H} \rangle - [\alpha_u]B$$

Range proofs: efficiency & extensions

Theorem

The **range proof protocol** Π_{rp} has *perfect completeness, computational extended witness emulation, perfect honest-verifier zero-knowledge*

Range proofs: efficiency & extensions

Theorem

The **range proof protocol** Π_{rp} has *perfect completeness, computational extended witness emulation, perfect honest-verifier zero-knowledge*

Note that protocol is efficient as it has logarithmic proof size: $2\log_2 n + 4$ elements of $\mathbb{G}$ and 5 elements of $\mathbb{F}_p$.

Range proofs: efficiency & extensions

Theorem

The **range proof protocol** Π_{rp} has *perfect completeness, computational extended witness emulation, perfect honest-verifier zero-knowledge*

Note that protocol is efficient as it has logarithmic proof size: $2\log_2 n + 4$ elements of $\mathbb{G}$ and 5 elements of $\mathbb{F}_p$.

Remark

The **range proof protocol** could be extended to support proving multiple range proofs at once with some efficiency improvements.

Range proofs & subset-sum NP-complete problem

One of the most famous *NP-complete* problems is the **subset-sum problem**: given a set of numbers presented as vector $\mathbf{s}$ and number $v \in \mathbb{N}$, does some subset sum up to v.

Range proofs & subset-sum NP-complete problem

One of the most famous *NP-complete* problems is the **subset-sum problem**: given a set of numbers presented as vector $\mathbf{s}$ and number $v \in \mathbb{N}$, does some subset sum up to v. It turns out that we could use our **range-proof** protocol for this problem. One could simply replace first inner-product check $\langle \mathbf{a}_L, \mathbf{2}^n \rangle = v$ with $\langle \mathbf{a}_L, \mathbf{s} \rangle = v$ where $\mathbf{a}_L$ is the secret vector of bits that encode positions of $\mathbf{s}$ that sum up to v.

Range proofs & subset-sum NP-complete problem

One of the most famous *NP-complete* problems is the **subset-sum problem**: given a set of numbers presented as vector $\mathbf{s}$ and number $v \in \mathbb{N}$, does some subset sum up to v. It turns out that we could use our **range-proof** protocol for this problem. One could simply replace first inner-product check $\langle \mathbf{a}_L, \mathbf{2}^n \rangle = v$ with $\langle \mathbf{a}_L, \mathbf{s} \rangle = v$ where $\mathbf{a}_L$ is the secret vector of bits that encode positions of $\mathbf{s}$ that sum up to v.

Example

Let $\mathbf{s} = (6, 8, 2, 3)$ and $v = 14$. Then setting $\mathbf{a}_L = (1, 1, 0, 0)$ we could use Π_{rp} to prove that there exists a subset of $\mathbf{s}$ that sums up to $v = 14$ without disclosing that subset.

Range proofs & subset-sum NP-complete problem

One of the most famous *NP-complete* problems is the **subset-sum problem**: given a set of numbers presented as vector $\mathbf{s}$ and number $v \in \mathbb{N}$, does some subset sum up to v. It turns out that we could use our **range-proof** protocol for this problem. One could simply replace first inner-product check $\langle \mathbf{a}_L, \mathbf{2}^n \rangle = v$ with $\langle \mathbf{a}_L, \mathbf{s} \rangle = v$ where $\mathbf{a}_L$ is the secret vector of bits that encode positions of $\mathbf{s}$ that sum up to v.

Example

Let $\mathbf{s} = (6, 8, 2, 3)$ and $v = 14$. Then setting $\mathbf{a}_L = (1, 1, 0, 0)$ we could use Π_{rp} to prove that there exists a subset of $\mathbf{s}$ that sums up to $v = 14$ without disclosing that subset.

Therefore, **bulletproofs range proof** protocol is capable to prove a knowledge of witness to any NP-problem as they all could be reduced to the **subset-sum problem**.

Arithmetic circuits

Bulletproofs for arithmetic circuits

- **Goal:** Prove that a circuit computes correctly without revealing inputs or intermediate values (*circuit satisfiability problem*).

Bulletproofs for arithmetic circuits

- **Goal:** Prove that a circuit computes correctly without revealing inputs or intermediate values (*circuit satisfiability problem*).
- **Approach:** Use inner-product argument to prove correctness of arithmetic circuits

Bulletproofs for arithmetic circuits

- **Goal:** Prove that a circuit computes correctly without revealing inputs or intermediate values (*circuit satisfiability problem*).
- **Approach:** Use inner-product argument to prove correctness of arithmetic circuits
- **Applications:** Privacy-preserving smart contracts, confidential computations, zero-knowledge proofs for complex computations

Bulletproofs for arithmetic circuits

- **Goal:** Prove that a circuit computes correctly without revealing inputs or intermediate values (*circuit satisfiability problem*).
- **Approach:** Use inner-product argument to prove correctness of arithmetic circuits
- **Applications:** Privacy-preserving smart contracts, confidential computations, zero-knowledge proofs for complex computations

Bulletproofs arithmetization slightly differs from the classic R1CS, however it could be transformed vice-versa easily. Also **bulletproofs** arithmetization is more convenient and human-friendly for encoding most of the arithmetic circuits than the R1CS.

Arithmetic circuits: variables

There are two types of variables in *bulletproofs* constraint system:

- **High-level variables** $\mathbf{v} \in \mathbb{F}_p^m$ are the private witness inputs to the circuit, typically provided with Pedersen commitments $\mathbf{V} \in \mathbb{G}^m$.

Arithmetic circuits: variables

There are two types of variables in *bulletproofs* constraint system:

- **High-level variables** $\mathbf{v} \in \mathbb{F}_p^m$ are the private witness inputs to the circuit, typically provided with Pedersen commitments $\mathbf{V} \in \mathbb{G}^m$.
- **Low-level variables** $\mathbf{a}_L, \mathbf{a}_R, \mathbf{a}_O \in \mathbb{F}_p^n$ are the intermediate witness values of computation.

Arithmetic circuits: variables

There are two types of variables in *bulletproofs* constraint system:

- **High-level variables** $\mathbf{v} \in \mathbb{F}_p^m$ are the private witness inputs to the circuit, typically provided with Pedersen commitments $\mathbf{V} \in \mathbb{G}^m$.
- **Low-level variables** $\mathbf{a}_L, \mathbf{a}_R, \mathbf{a}_O \in \mathbb{F}_p^n$ are the intermediate witness values of computation.

We will define circuit as a set of multiplication constraints operating with *low-level* variables and set of linear constraints which links *low-level* variables between each other and *high-level* variables as well.

Arithmetic circuits: constraints

Multiplication constraints are defined with one vector equation:

$$\mathbf{a}_L \circ \mathbf{a}_R = \mathbf{a}_O$$

Arithmetic circuits: constraints

Multiplication constraints are defined with one vector equation:

$$\mathbf{a}_L \circ \mathbf{a}_R = \mathbf{a}_O$$

Linear constraints are defined via:

$$\mathbf{W}_L \cdot \mathbf{a}_L + \mathbf{W}_R \cdot \mathbf{a}_R + \mathbf{W}_O \cdot \mathbf{a}_O = \mathbf{W}_V \cdot \mathbf{v} + \mathbf{c}$$

Arithmetic circuits: constraints

Multiplication constraints are defined with one vector equation:

$$\mathbf{a}_L \circ \mathbf{a}_R = \mathbf{a}_O$$

Linear constraints are defined via:

$$\mathbf{W}_L \cdot \mathbf{a}_L + \mathbf{W}_R \cdot \mathbf{a}_R + \mathbf{W}_O \cdot \mathbf{a}_O = \mathbf{W}_V \cdot \mathbf{v} + \mathbf{c}$$

Where $\mathbf{a_L}, \mathbf{a_R}, \mathbf{a_O}$ – vectors of left and right inputs for multiplication gates and output values (all of them are low-level variables). $\mathbf{W_L}, \mathbf{W_R}, \mathbf{W_O} \in \mathbb{F}_p^{q \times n}, \mathbf{W}_V \in \mathbb{F}_p^{q \times m}$ – public matrices of weights for linear constraints (obviously known to verifier). $\mathbf{c} \in \mathbb{F}_p^q$ – public vector of constants. Typically they encode wiring of the circuit and other linear relations between variables.

Arithmetic circuits: example

Example

Consider the following elliptic curve membership circuit. Here witness (v_1, v_2) should satisfy elliptic curve equation:

$$y^2 = x^3 + ax + b$$

The arithmetization for this circuit is as follows:

Arithmetic circuits: example

Example

Consider the following elliptic curve membership circuit. Here witness (v_1, v_2) should satisfy elliptic curve equation:

$$y^2 = x^3 + ax + b$$

The arithmetization for this circuit is as follows:

Low-level variables:

$$\mathbf{a}_L = \begin{bmatrix} x \\ x \\ y \end{bmatrix}, \quad \mathbf{a}_R = \begin{bmatrix} x \\ x^2 \\ y \end{bmatrix}, \quad \mathbf{a}_O = \begin{bmatrix} x^2 \\ x^3 \\ y^2 \end{bmatrix}$$

Arithmetic circuits: example

Example

Consider the following elliptic curve membership circuit. Here witness (v_1, v_2) should satisfy elliptic curve equation:

$$y^2 = x^3 + ax + b$$

The arithmetization for this circuit is as follows:

Low-level variables:

$$\mathbf{a}_L = \begin{bmatrix} x \\ x \\ y \end{bmatrix}, \quad \mathbf{a}_R = \begin{bmatrix} x \\ x^2 \\ y \end{bmatrix}, \quad \mathbf{a}_O = \begin{bmatrix} x^2 \\ x^3 \\ y^2 \end{bmatrix}$$

High-level variables:

$$\mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \end{bmatrix}$$

Arithmetic circuits: example

Example

Multiplication constraints:

$$\mathbf{a}_L \circ \mathbf{a}_R = \mathbf{a}_O \Rightarrow \begin{bmatrix} x \cdot x = x^2 \\ x \cdot x^2 = x^3 \\ y \cdot y = y^2 \end{bmatrix}$$

Arithmetic circuits: example

Example

Multiplication constraints:

$$\mathbf{a}_L \circ \mathbf{a}_R = \mathbf{a}_O \Rightarrow \begin{bmatrix} x \cdot x = x^2 \\ x \cdot x^2 = x^3 \\ y \cdot y = y^2 \end{bmatrix}$$

Linear constraints:

$$\begin{aligned} \mathbf{a}_L^{(1)} &= v_1 & \mathbf{a}_R^{(1)} &= v_1 \\ \mathbf{a}_L^{(2)} - \mathbf{a}_L^{(1)} &= 0 & \mathbf{a}_R^{(2)} - \mathbf{a}_O^{(1)} &= 0 \\ \mathbf{a}_L^{(3)} &= v_2 & \mathbf{a}_R^{(3)} &= v_2 \\ \mathbf{a}_O^{(3)} - \mathbf{a}_O^{(2)} - a \cdot \mathbf{a}_L^{(1)} &= b \end{aligned}$$

Arithmetic circuits: example

Example

Multiplication constraints:

$$\mathbf{a}_L \circ \mathbf{a}_R = \mathbf{a}_O \Rightarrow \begin{bmatrix} x \cdot x = x^2 \\ x \cdot x^2 = x^3 \\ y \cdot y = y^2 \end{bmatrix}$$

Linear constraints:

$$\begin{aligned} \mathbf{a}_L^{(1)} &= v_1 & \mathbf{a}_R^{(1)} &= v_1 \\ \mathbf{a}_L^{(2)} - \mathbf{a}_L^{(1)} &= 0 & \mathbf{a}_R^{(2)} - \mathbf{a}_O^{(1)} &= 0 \\ \mathbf{a}_L^{(3)} &= v_2 & \mathbf{a}_R^{(3)} &= v_2 \\ \mathbf{a}_O^{(3)} - \mathbf{a}_O^{(2)} - a \cdot \mathbf{a}_L^{(1)} &= b \end{aligned}$$

$$\mathbf{W}_L \cdot \mathbf{a}_L + \mathbf{W}_R \cdot \mathbf{a}_R + \mathbf{W}_O \cdot \mathbf{a}_O = \mathbf{W}_V \cdot \mathbf{v} + \mathbf{c}$$

Arithmetic circuits: example

Example

$$\mathbf{W}_L = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ -1 & 1 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \\ -a & 0 & 0 \end{bmatrix}, \quad \mathbf{W}_R = \begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}, \quad \mathbf{W}_O = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & -1 & 1 \end{bmatrix},$$

$$\mathbf{W}_V = \begin{bmatrix} 1 & 0 \\ 1 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 1 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}, \quad \mathbf{c} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ b \end{bmatrix}$$

Bulletproofs for circuits: relation

Consider the relation:

$$\mathcal{R}_{sat} = \left\{ \begin{array}{l} (G, B, \mathbf{V}, \mathbf{W}_L, \mathbf{W}_R, \mathbf{W}_O, \mathbf{W}_V, \mathbf{c}; \mathbf{a}_L, \mathbf{a}_R, \mathbf{a}_O, \mathbf{v}, \mathbf{r}) : \\ \forall i \in \{1, \ldots, m\} : V_i = [v_i]G + [r_i]B \wedge \\ \mathbf{a}_L \circ \mathbf{a}_R = \mathbf{a}_O \wedge \\ \mathbf{W}_L \cdot \mathbf{a}_L + \mathbf{W}_R \cdot \mathbf{a}_R + \mathbf{W}_O \cdot \mathbf{a}_O = \mathbf{W}_V \cdot \mathbf{v} + \mathbf{c} \end{array} \right\}$$

Where $\mathbf{a}_L, \mathbf{a}_R, \mathbf{a}_O \in \mathbb{F}_p^n$, $\mathbf{v}, \mathbf{r} \in \mathbb{F}_p^m$, $\mathbf{W}_L, \mathbf{W}_R, \mathbf{W}_O \in \mathbb{F}_p^{q \times n}$, $\mathbf{W}_V \in \mathbb{F}_p^{q \times m}$, $\mathbf{c} \in \mathbb{F}_p^q$.

Bulletproofs for circuits: relation

Consider the relation:

$$\mathcal{R}_{sat} = \left\{ \begin{array}{l} (G, B, \mathbf{V}, \mathbf{W}_L, \mathbf{W}_R, \mathbf{W}_O, \mathbf{W}_V, \mathbf{c}; \mathbf{a}_L, \mathbf{a}_R, \mathbf{a}_O, \mathbf{v}, \mathbf{r}) : \\ \forall i \in \{1, \ldots, m\} : V_i = [v_i]G + [r_i]B \wedge \\ \mathbf{a}_L \circ \mathbf{a}_R = \mathbf{a}_O \wedge \\ \mathbf{W}_L \cdot \mathbf{a}_L + \mathbf{W}_R \cdot \mathbf{a}_R + \mathbf{W}_O \cdot \mathbf{a}_O = \mathbf{W}_V \cdot \mathbf{v} + \mathbf{c} \end{array} \right\}$$

Where $\mathbf{a}_L, \mathbf{a}_R, \mathbf{a}_O \in \mathbb{F}_p^n$, $\mathbf{v}, \mathbf{r} \in \mathbb{F}_p^m$, $\mathbf{W}_L, \mathbf{W}_R, \mathbf{W}_O \in \mathbb{F}_p^{q \times n}$, $\mathbf{W}_V \in \mathbb{F}_p^{q \times m}$, $\mathbf{c} \in \mathbb{F}_p^q$.

Note

Informally this relation states that there exists a valid witness $\mathbf{v}$ that satisfies all constraints of the circuit. For the verifier witness is presented only as commitments vector $\mathbf{V}$.

Arithmetic circuits: compiling into inner-product

We could use similar to *range-proofs* technique to compile constraints of the circuit into inner-product relation. For multiplicative constraints take random $y \in \mathbb{F}_p$ and apply zero check:

$$\langle \mathbf{a}_L \circ \mathbf{a}_R - \mathbf{a}_O, \mathbf{y}^n \rangle = 0$$

Arithmetic circuits: compiling into inner-product

We could use similar to *range-proofs* technique to compile constraints of the circuit into inner-product relation. For multiplicative constraints take random $y \in \mathbb{F}_p$ and apply zero check:

$$\langle \mathbf{a}_L \circ \mathbf{a}_R - \mathbf{a}_O, \mathbf{y}^n \rangle = 0$$

Same for linear constraints, but for different randomness $z \in \mathbb{F}_p$:

$$\langle \mathbf{W}_L \cdot \mathbf{a}_L + \mathbf{W}_R \cdot \mathbf{a}_R + \mathbf{W}_O \cdot \mathbf{a}_O - \mathbf{W}_V \cdot \mathbf{v} - \mathbf{c}, \mathbf{z}^q \rangle = 0$$

Arithmetic circuits: compiling into inner-product

We could use similar to *range-proofs* technique to compile constraints of the circuit into inner-product relation. For multiplicative constraints take random $y \in \mathbb{F}_p$ and apply zero check:

$$\langle \mathbf{a}_L \circ \mathbf{a}_R - \mathbf{a}_O, \mathbf{y}^n \rangle = 0$$

Same for linear constraints, but for different randomness $z \in \mathbb{F}_p$:

$$\langle \mathbf{W}_L \cdot \mathbf{a}_L + \mathbf{W}_R \cdot \mathbf{a}_R + \mathbf{W}_O \cdot \mathbf{a}_O - \mathbf{W}_V \cdot \mathbf{v} - \mathbf{c}, \mathbf{z}^q \rangle = 0$$

Combine these two checks to one using the same randomness z:

$$\langle \mathbf{a}_L \circ \mathbf{a}_R - \mathbf{a}_O, \mathbf{y}^n \rangle + \langle z \cdot \mathbf{z}^q, \mathbf{W}_L \cdot \mathbf{a}_L + \mathbf{W}_R \cdot \mathbf{a}_R + \mathbf{W}_O \cdot \mathbf{a}_O - \mathbf{W}_V \cdot \mathbf{v} - \mathbf{c} \rangle = 0$$

This check is sound as typically a prover could not control values of y, z before he commits to $\mathbf{a_L}, \mathbf{a_R}, \mathbf{a_O}$ and $\mathbf{v}$.

Arithmetic circuits: compiling into inner-product

Denote $w_c = \langle z \cdot \mathbf{z}^q, \mathbf{c} \rangle$ and flattened linear constraints (still public and easily computed by verifier):

$$\mathbf{w}_L = \mathbf{W}_L^T \cdot (z \cdot \mathbf{z}^q) \quad \mathbf{w}_R = \mathbf{W}_R^T \cdot (z \cdot \mathbf{z}^q)$$
$$\mathbf{w}_O = \mathbf{W}_O^T \cdot (z \cdot \mathbf{z}^q) \quad \mathbf{w}_V = \mathbf{W}_V^T \cdot (z \cdot \mathbf{z}^q)$$

Arithmetic circuits: compiling into inner-product

Denote $w_c = \langle z \cdot \mathbf{z}^q, \mathbf{c} \rangle$ and flattened linear constraints (still public and easily computed by verifier):

$$\mathbf{w}_L = \mathbf{W}_L^T \cdot (z \cdot \mathbf{z}^q) \quad \mathbf{w}_R = \mathbf{W}_R^T \cdot (z \cdot \mathbf{z}^q)$$
$$\mathbf{w}_O = \mathbf{W}_O^T \cdot (z \cdot \mathbf{z}^q) \quad \mathbf{w}_V = \mathbf{W}_V^T \cdot (z \cdot \mathbf{z}^q)$$

Again doing some linear algebra witchcraft we could separate $\mathbf{a}_L, \mathbf{a}_O$ to be on the left side of the inner-product and $\mathbf{a}_R$ to be on the right:

$$w_c + \langle \mathbf{w}_V, \mathbf{v} \rangle + \delta(y, z) =$$
$$\langle \mathbf{a}_L + \mathbf{y}^{-n} \circ \mathbf{w}_R, \mathbf{y}^n \circ \mathbf{a}_R + \mathbf{w}_L \rangle + \langle \mathbf{a}_O, -\mathbf{y}^n + \mathbf{w}_O \rangle$$

Where $\delta(y, z) = \langle \mathbf{y}^{-n} \circ \mathbf{w}_R, \mathbf{w}_L \rangle$ – easily computable by $\mathcal{V}$.

Arithmetic circuits: compiling into inner-product

Denote $w_c = \langle z \cdot \mathbf{z}^q, \mathbf{c} \rangle$ and flattened linear constraints (still public and easily computed by verifier):

$$\mathbf{w}_L = \mathbf{W}_L^T \cdot (z \cdot \mathbf{z}^q) \quad \mathbf{w}_R = \mathbf{W}_R^T \cdot (z \cdot \mathbf{z}^q)$$
$$\mathbf{w}_O = \mathbf{W}_O^T \cdot (z \cdot \mathbf{z}^q) \quad \mathbf{w}_V = \mathbf{W}_V^T \cdot (z \cdot \mathbf{z}^q)$$

Again doing some linear algebra witchcraft we could separate $\mathbf{a}_L, \mathbf{a}_O$ to be on the left side of the inner-product and $\mathbf{a}_R$ to be on the right:

$$w_c + \langle \mathbf{w}_V, \mathbf{v} \rangle + \delta(y, z) =$$
$$\langle \mathbf{a}_L + \mathbf{y}^{-n} \circ \mathbf{w}_R, \mathbf{y}^n \circ \mathbf{a}_R + \mathbf{w}_L \rangle + \langle \mathbf{a}_O, -\mathbf{y}^n + \mathbf{w}_O \rangle$$

Where $\delta(y, z) = \langle \mathbf{y}^{-n} \circ \mathbf{w}_R, \mathbf{w}_L \rangle$ – easily computable by $\mathcal{V}$.

Here we have a sum of 2 separate inner-products, we could express it as second-degree coefficient of the following polynomial:

$$\langle \mathbf{a}x + \mathbf{c}x^2, \mathbf{d} + \mathbf{b}x \rangle = s_1 x + s_2 x^2 + s_3 x^3 =$$
$$x \cdot \langle \mathbf{a}, \mathbf{d} \rangle + x^2 \cdot (\langle \mathbf{a}, \mathbf{b} \rangle + \langle \mathbf{c}, \mathbf{d} \rangle) + x^3 \cdot \langle \mathbf{c}, \mathbf{b} \rangle$$

Arithmetic circuits: compiling into inner-product

$$\mathbf{a} \leftarrow \mathbf{a}_L + \mathbf{y}^{-n} \circ \mathbf{w}_R \qquad \mathbf{b} \leftarrow \mathbf{y}^n \circ \mathbf{a}_R + \mathbf{w}_L$$

$$\mathbf{c} \leftarrow \mathbf{a}_O \qquad \mathbf{d} \leftarrow -\mathbf{y}^n + \mathbf{w}_O$$

Desired sum of inner products is the second-degree coefficient s_2:

$$w_c + \langle \mathbf{w}_V, \mathbf{v} \rangle + \delta(y, z) = s_2$$

Arithmetic circuits: compiling into inner-product

$$\mathbf{a} \leftarrow \mathbf{a}_L + \mathbf{y}^{-n} \circ \mathbf{w}_R \quad \mathbf{b} \leftarrow \mathbf{y}^n \circ \mathbf{a}_R + \mathbf{w}_L$$

$$\mathbf{c} \leftarrow \mathbf{a}_O \quad \mathbf{d} \leftarrow -\mathbf{y}^n + \mathbf{w}_O$$

Desired sum of inner products is the second-degree coefficient s_2:

$$w_c + \langle \mathbf{w}_V, \mathbf{v} \rangle + \delta(y, z) = s_2$$

To obtain final polynomials $\mathbf{l}(x), \mathbf{r}(x)$ we must firstly blind $\mathbf{a}_L, \mathbf{a}_R$:

$$\mathbf{a}_L \leftarrow \mathbf{a}_L + \mathbf{s}_L x^2 \quad \mathbf{a}_R \leftarrow \mathbf{a}_R + \mathbf{s}_R x^2$$

Arithmetic circuits: compiling into inner-product

$$\mathbf{a} \leftarrow \mathbf{a}_L + \mathbf{y}^{-n} \circ \mathbf{w}_R \quad \mathbf{b} \leftarrow \mathbf{y}^n \circ \mathbf{a}_R + \mathbf{w}_L$$
$$\mathbf{c} \leftarrow \mathbf{a}_O \quad \mathbf{d} \leftarrow -\mathbf{y}^n + \mathbf{w}_O$$

Desired sum of inner products is the second-degree coefficient s_2:

$$w_c + \langle \mathbf{w}_V, \mathbf{v} \rangle + \delta(y, z) = s_2$$

To obtain final polynomials $\mathbf{l}(x), \mathbf{r}(x)$ we must firstly blind $\mathbf{a}_L, \mathbf{a}_R$:

$$\mathbf{a}_L \leftarrow \mathbf{a}_L + \mathbf{s}_L x^2 \quad \mathbf{a}_R \leftarrow \mathbf{a}_R + \mathbf{s}_R x^2$$

And finally compute polynomials $\mathbf{l}(x), \mathbf{r}(x)$ as follows:

$$\mathbf{l}(x) = \mathbf{s}_L \cdot x^3 + \mathbf{a}_O \cdot x^2 + (\mathbf{a}_L + \mathbf{y}^{-n} \circ \mathbf{w}_R) \cdot x$$
$$\mathbf{r}(x) = \mathbf{y}^n \circ \mathbf{s}_R \cdot x^3 + (\mathbf{y}^n \circ \mathbf{a}_R + \mathbf{w}_L) \cdot x - \mathbf{y}^n + \mathbf{w}_O$$
$$t(x) = \langle \mathbf{l}(x), \mathbf{r}(x) \rangle = \sum_{i=1}^{6} t_i x^i$$

Where $t_2 = w_c + \langle \mathbf{w}_V, \mathbf{v} \rangle + \delta(y, z)$ – desired sum of inner-products.

Arithmetic circuits: witness commitments

Here we could again apply modified **zk-mul** to prove that t_2 is a valid sum of inner-products:

- Setup: returns vectors of independent generators $\mathbf{G}, \mathbf{H} \in \mathbb{G}^n$.

Arithmetic circuits: witness commitments

Here we could again apply modified **zk-mul** to prove that t_2 is a valid sum of inner-products:

- Setup: returns vectors of independent generators $\mathbf{G}, \mathbf{H} \in \mathbb{G}^n$.
- Prover $\mathcal{P}$ chooses blinding factors $\alpha, \beta, \gamma \in \mathbb{F}_p, \mathbf{s}_L, \mathbf{s}_R \in \mathbb{F}_p^n$ and sends the following commitments to $\mathcal{V}$:

$$
\begin{aligned}
A_I &= \langle \mathbf{a}_L, \mathbf{G} \rangle + \langle \mathbf{a}_R, \mathbf{H} \rangle + [\alpha]B \\
A_O &= \langle \mathbf{a}_O, \mathbf{G} \rangle + [\gamma]B \\
S &= \langle \mathbf{s}_L, \mathbf{G} \rangle + \langle \mathbf{s}_R, \mathbf{H} \rangle + [\beta]B
\end{aligned}
$$

Arithmetic circuits: witness commitments

Here we could again apply modified **zk-mul** to prove that t_2 is a valid sum of inner-products:

- Setup: returns vectors of independent generators $\mathbf{G}, \mathbf{H} \in \mathbb{G}^n$.
- Prover $\mathcal{P}$ chooses blinding factors $\alpha, \beta, \gamma \in \mathbb{F}_p, \mathbf{s}_L, \mathbf{s}_R \in \mathbb{F}_p^n$ and sends the following commitments to $\mathcal{V}$:

$$
\begin{aligned}
A_I &= \langle \mathbf{a}_L, \mathbf{G} \rangle + \langle \mathbf{a}_R, \mathbf{H} \rangle + [\alpha]B \\
A_O &= \langle \mathbf{a}_O, \mathbf{G} \rangle + [\gamma]B \\
S &= \langle \mathbf{s}_L, \mathbf{G} \rangle + \langle \mathbf{s}_R, \mathbf{H} \rangle + [\beta]B
\end{aligned}
$$

- Verifier samples challenges $y, z \leftarrow\!\$\ \mathbb{F}_p$ and sends them to $\mathcal{P}$.

Arithmetic circuits: product commitments

- Using challenges y, z prover forms polynomials $\mathbf{l}(x), \mathbf{r}(x), t(x)$:

$$\mathbf{l}(x) = \mathbf{s}_L \cdot x^3 + \mathbf{a}_O \cdot x^2 + (\mathbf{a}_L + \mathbf{y}^{-n} \circ \mathbf{w}_R) \cdot x$$

$$\mathbf{r}(x) = \mathbf{y}^n \circ \mathbf{s}_R \cdot x^3 + (\mathbf{y}^n \circ \mathbf{a}_R + \mathbf{w}_L) \cdot x - \mathbf{y}^n + \mathbf{w}_O$$

$$t(x) = \langle \mathbf{l}(x), \mathbf{r}(x) \rangle = t_1 x + t_2 x^2 + t_3 x^3 + t_4 x^4 + t_5 x^5 + t_6 x^6$$

Arithmetic circuits: product commitments

- Using challenges y, z prover forms polynomials $\mathbf{l}(x), \mathbf{r}(x), t(x)$:

$$\mathbf{l}(x) = \mathbf{s}_L \cdot x^3 + \mathbf{a}_O \cdot x^2 + (\mathbf{a}_L + \mathbf{y}^{-n} \circ \mathbf{w}_R) \cdot x$$
$$\mathbf{r}(x) = \mathbf{y}^n \circ \mathbf{s}_R \cdot x^3 + (\mathbf{y}^n \circ \mathbf{a}_R + \mathbf{w}_L) \cdot x - \mathbf{y}^n + \mathbf{w}_O$$
$$t(x) = \langle \mathbf{l}(x), \mathbf{r}(x) \rangle = t_1 x + t_2 x^2 + t_3 x^3 + t_4 x^4 + t_5 x^5 + t_6 x^6$$

$\mathcal{P}$ chooses random blinding factors $\tau_1, \tau_3, \tau_4, \tau_5, \tau_6 \in \mathbb{F}_p$ and sends to $\mathcal{V}$ commitments to its coefficients:

$$T_1 = [t_1]G + [\tau_1]B \quad T_3 = [t_3]G + [\tau_3]B \quad T_4 = [t_4]G + [\tau_4]B$$
$$T_5 = [t_5]G + [\tau_5]B \quad T_6 = [t_6]G + [\tau_6]B$$

Arithmetic circuits: product commitments

- Using challenges y, z prover forms polynomials $\mathbf{l}(x), \mathbf{r}(x), t(x)$:

$$\mathbf{l}(x) = \mathbf{s}_L \cdot x^3 + \mathbf{a}_O \cdot x^2 + (\mathbf{a}_L + \mathbf{y}^{-n} \circ \mathbf{w}_R) \cdot x$$
$$\mathbf{r}(x) = \mathbf{y}^n \circ \mathbf{s}_R \cdot x^3 + (\mathbf{y}^n \circ \mathbf{a}_R + \mathbf{w}_L) \cdot x - \mathbf{y}^n + \mathbf{w}_O$$
$$t(x) = \langle \mathbf{l}(x), \mathbf{r}(x) \rangle = t_1 x + t_2 x^2 + t_3 x^3 + t_4 x^4 + t_5 x^5 + t_6 x^6$$

$\mathcal{P}$ chooses random blinding factors $\tau_1, \tau_3, \tau_4, \tau_5, \tau_6 \in \mathbb{F}_p$ and sends to $\mathcal{V}$ commitments to its coefficients:

$$T_1 = [t_1]G + [\tau_1]B \quad T_3 = [t_3]G + [\tau_3]B \quad T_4 = [t_4]G + [\tau_4]B$$
$$T_5 = [t_5]G + [\tau_5]B \quad T_6 = [t_6]G + [\tau_6]B$$

Note: Prover does not send separate commitment to t_2 as the verifier could derive it from $\mathbf{V}$ and the circuit public parameters:

$$t_2 = w_c + \langle \mathbf{w}_V, \mathbf{v} \rangle + \delta(y, z)$$
$$T_2 = \langle \mathbf{w}_V, \mathbf{V} \rangle + [\delta(y, z) + w_c]G$$

Arithmetic circuits: evaluating polynomials

- Verifier samples and sends to $\mathcal{P}$ random evaluation point $u \leftarrow\$ \mathbb{F}_p$.

Arithmetic circuits: evaluating polynomials

- Verifier samples and sends to $\mathcal{P}$ random evaluation point $u \leftarrow\$ \mathbb{F}_p$.
- Prover evaluates polynomials at u:

$$
\begin{aligned}
\mathbf{l}_u &= \mathbf{l}(u) \\
\mathbf{r}_u &= \mathbf{r}(u) \\
t_u &= \langle \mathbf{l}_u, \mathbf{r}_u \rangle = t(u) \\
\tau_u &= \tau_1 \cdot u + \langle \mathbf{w}_V, \mathbf{r} \rangle u^2 + \tau_3 \cdot u^3 + \tau_4 \cdot u^4 + \tau_5 \cdot u^5 + \tau_6 \cdot u^6 \\
\alpha_u &= \alpha u + \gamma u^2 + \beta u^3
\end{aligned}
$$

and sends $(\mathbf{l}_u, \mathbf{r}_u, t_u, \alpha_u, \tau_u)$ to $\mathcal{V}$.

Arithmetic circuits: verification

- Verifier performs checks:

$$
\begin{aligned}
&[u]A_I + [u^2]A_O + [u^3]S - \langle \mathbf{1}, \mathbf{H} \rangle + \\
&u \cdot (\langle \mathbf{y}^{-n} \circ \mathbf{w}_L, \mathbf{G} \rangle + \langle \mathbf{y}^{-n} \circ \mathbf{w}_R, \mathbf{H} \rangle) + \langle \mathbf{y}^{-n} \circ \mathbf{w}_O, \mathbf{H} \rangle \\
&\stackrel{?}{=} \langle \mathbf{l}_u, \mathbf{G} \rangle + \langle \mathbf{r}_u, \mathbf{y}^{-n} \circ \mathbf{H} \rangle + [\alpha_u]B \\
[t_u]G + [\tau_u]B &\stackrel{?}{=} [u]T_1 + u^2 \cdot (\langle \mathbf{w}_V, \mathbf{V} \rangle + [\delta(y,z) + w_c]G) + \\
&[u^3]T_3 + [u^4]T_4 + [u^5]T_5 + [u^6]T_6 \\
t_u &\stackrel{?}{=} \langle \mathbf{l}_u, \mathbf{r}_u \rangle
\end{aligned}
$$

Arithmetic circuits: verification

- Verifier performs checks:

$$
\begin{aligned}
&[u]A_I + [u^2]A_O + [u^3]S - \langle \mathbf{1}, \mathbf{H} \rangle + \\
&u \cdot (\langle \mathbf{y}^{-n} \circ \mathbf{w}_L, \mathbf{G} \rangle + \langle \mathbf{y}^{-n} \circ \mathbf{w}_R, \mathbf{H} \rangle) + \langle \mathbf{y}^{-n} \circ \mathbf{w}_O, \mathbf{H} \rangle \\
&\stackrel{?}{=} \langle \mathbf{l}_u, \mathbf{G} \rangle + \langle \mathbf{r}_u, \mathbf{y}^{-n} \circ \mathbf{H} \rangle + [\alpha_u]B
\end{aligned}
$$

$$
\begin{aligned}
[t_u]G + [\tau_u]B \stackrel{?}{=}\ & [u]T_1 + u^2 \cdot (\langle \mathbf{w}_V, \mathbf{V} \rangle + [\delta(y,z) + w_c]G) + \\
& [u^3]T_3 + [u^4]T_4 + [u^5]T_5 + [u^6]T_6
\end{aligned}
$$

$$
t_u \stackrel{?}{=} \langle \mathbf{l}_u, \mathbf{r}_u \rangle
$$

Remark

To provide logarithmic proof instead of sending $\mathbf{l}_u, \mathbf{r}_u$ parties could run **IPA** on inputs $(\mathbf{G}, \mathbf{y}^{-n} \circ \mathbf{H}, P, t_u; \mathbf{l}_u, \mathbf{r}_u)$ where:

$$
\begin{aligned}
P =& [u]A_I + [u^2]A_O + [u^3]S - \langle \mathbf{1}, \mathbf{H} \rangle + \\
& u \cdot (\langle \mathbf{y}^{-n} \circ \mathbf{w}_L, \mathbf{G} \rangle + \langle \mathbf{y}^{-n} \circ \mathbf{w}_R, \mathbf{H} \rangle) + \langle \mathbf{y}^{-n} \circ \mathbf{w}_O, \mathbf{H} \rangle - [\alpha_u]B
\end{aligned}
$$

Arithmetic circuits: efficiency & extensions

Theorem

The **arithmetic circuits protocol** Π_{sat} has *perfect completeness, computational extended witness emulation, perfect honest-verifier zero-knowledge*

Arithmetic circuits: efficiency & extensions

Theorem

The **arithmetic circuits protocol** Π_{sat} has *perfect completeness, computational extended witness emulation, perfect honest-verifier zero-knowledge*

The protocol is efficient as it has logarithmic proof size: $2\log_2 n + 8$ elements of $\mathbb{G}$ and 5 elements of $\mathbb{F}_p$.

Arithmetic circuits: efficiency & extensions

Theorem

The **arithmetic circuits protocol** Π_{sat} has *perfect completeness, computational extended witness emulation, perfect honest-verifier zero-knowledge*

The protocol is efficient as it has logarithmic proof size: $2\log_2 n + 8$ elements of $\mathbb{G}$ and 5 elements of $\mathbb{F}_p$.

Remark

The **arithmetic circuits protocol** could be slightly modified to provide intermediate random challenges inside the circuit. For example it would allow proving *permutation check*:
$\{a, b\} = \{c, d\} \iff (a - x) \cdot (b - x) = (c - x) \cdot (d - x)$ for some random challenge x.

Thank you for your attention



research.blockstream.com

github.com/BlockstreamResearch/zk-book



Blockstream